

1. Summary

1.1 Email Notification System Implementation

During this period, work focused on implementing and validating an automated email notification system for the Eco Clean platform. The goal was to ensure that users receive timely reminders for scheduled appointments.

Backend logic was implemented to trigger reminder emails based on predefined scheduling conditions. Scheduled background tasks evaluate upcoming appointments and send notifications when specific time thresholds are reached. This allows reminders to be delivered automatically without requiring manual intervention from administrators.

Testing was conducted to confirm that reminders are triggered correctly across different scheduling scenarios. This improves operational communication and ensures that both staff members and clients receive timely updates regarding upcoming appointments.

1.2 Database Seeding and Test Data Preparation

Another major focus during this period was preparing the system for testing through structured database seeding. Seed scripts were created to automatically generate realistic development data including users, staff members, clients, and appointment records.

This approach allows the system to be tested with datasets that better resemble real-world usage. By generating structured sample data, development and testing workflows become more efficient and system features can be validated across multiple scenarios.

The seeded dataset also supports future testing and demonstrations by providing consistent and repeatable testing conditions.

1.3 System Stability and Deployment Improvements

Additional work focused on improving the stability and reliability of backend services. Several issues related to API responses, database interactions, and environment configuration were identified and resolved.

These improvements included refining API endpoints, ensuring consistent data handling between frontend and backend components, and resolving configuration issues that affected system behavior during development.

Addressing these issues helps reduce unexpected errors and improves overall system reliability.

1.4 Calendar Filters Implementation

During this milestone, calendar filtering functionality was implemented to improve the usability of the scheduling interface within the Eco Clean platform. As the number of scheduled appointments increases, filtering becomes essential for administrators to quickly locate relevant bookings and manage daily operations efficiently.

The calendar interface was extended to support filtering based on key scheduling attributes such as appointment status, assigned staff members, and related entities. These filters allow users to dynamically refine the visible appointments within the calendar view without needing to reload or navigate away from the scheduling interface.

Backend query logic was updated to support these filtering conditions, ensuring that only the relevant appointment data is retrieved and displayed. The frontend calendar components were also updated to react to filter changes and refresh the displayed events accordingly.

This improvement significantly enhances the usability of the scheduling system by allowing administrators to focus on specific subsets of appointments, such as upcoming tasks, completed jobs, or assignments related to particular staff members. The addition of calendar filters makes the scheduling interface more scalable and easier to manage as the system grows.

2. Technical Decisions Taken

2.1 Scheduled Dispatch Time for Reminder Emails

A decision was made to dispatch reminder emails at a fixed daily time of 9:00 AM. Instead of triggering reminders immediately when conditions are met, the system performs a scheduled daily check of upcoming appointments and sends reminders when required.

This approach simplifies scheduling logic, ensures consistent notification timing, and reduces unnecessary server load.

2.2 Support for Multiple Reminder Intervals

The reminder system was designed to support multiple reminder intervals before an appointment occurs. Rather than relying on a single notification trigger, the system allows reminders to be sent at different predefined time thresholds.

This structure improves flexibility and allows additional reminder stages to be introduced in the future if needed.

2.3 Reminder State Tracking in the Database

Database-level flags were introduced to track whether a reminder has already been sent. Each appointment record maintains fields that indicate the status of each reminder stage. This ensures that reminders are sent only once, even if the scheduled reminder process runs multiple times.

2.4 Cron-Based Background Job for Reminder Processing

Reminder processing was implemented using a cron-based scheduled endpoint that evaluates upcoming appointments and executes reminder logic when necessary. This provides a simple and maintainable way to run background tasks without requiring a dedicated worker service.

3. Overall Progress

Overall, the work completed during this period focused on strengthening the reliability and testing infrastructure of the Eco Clean platform.

Key improvements included implementing automated email reminders, introducing structured database seed scripts, and improving backend stability. System testing confirmed that scheduling workflows, notification triggers, and seeded datasets integrate correctly within the existing architecture.

These improvements enhance the platform’s operational reliability and support continued development.

4. Work Logs

Date	Time	Description
2026-03-08	10:00–11:30am	Designed backend reminder architecture and planned automated email notification workflow
2026-03-08	11:30am–1:00pm	Implemented initial reminder logic to evaluate upcoming appointments and determine notification thresholds
2026-03-08	2:00–3:30pm	Developed email dispatch functionality and integrated email service with appointment reminder logic
2026-03-08	4:00–5:30pm	Tested reminder triggers using sample appointment data and validated correct email delivery behavior
2026-03-09	9:30–11:00am	Implemented scheduled reminder endpoint and configured cron-based reminder processing
2026-03-09	11:00am–12:30pm	Added database flags to track reminder states and prevent duplicate notifications
2026-03-09	2:00–3:30pm	Implemented support for multiple reminder intervals before appointment time
2026-03-09	4:00–5:30pm	Debugged reminder timing logic and verified correct evaluation of appointment time windows
2026-03-10	9:30–11:00am	Developed database seed scripts to generate users, clients, staff, and appointment test data
2026-03-10	11:00am–12:30pm	Generated large seed datasets to simulate realistic system usage scenarios
2026-03-10	2:00–3:30pm	Integrated seeded data with scheduling and reminder workflows for testing
2026-03-10	4:00–5:00pm	Tested seeded data behavior across scheduling, notification, and role-based access features

2026-03-11	9:30–11:00am	Resolved backend stability issues related to API responses and database interactions
2026-03-11	11:00am–12:30pm	Refined environment configuration and improved backend error handling
2026-03-11	2:00–3:30pm	Validated reminder cron execution and confirmed email delivery across multiple test scenarios
2026-03-11	4:00–5:00pm	Performed end-to-end testing of scheduling workflow, reminders, and seeded data integration
2026-03-12	10:00–11:30am	Conducted additional system validation and confirmed stability of reminder dispatch system
2026-03-12	11:30am–12:30pm	Reviewed backend logic and optimized reminder evaluation queries
2026-03-12	2:00–3:30pm	Documented implementation details and prepared weekly progress report content
2026-03-13	9:30–11:00am	Tested reminder scheduling edge cases and verified correct time window evaluation
2026-03-13	11:00am–12:00pm	Refined seed data generation and improved test dataset consistency
2026-03-14	10:00–11:00am	Final verification of reminder emails, cron configuration, and seeded data behavior
Total	24h	

5. AI Usage

AI	Prompt	Value Addition
ChatGPT 5.3, Plus	Designing automated email reminder system for scheduled appointments	Helped structure reminder architecture and scheduling logic for automated notifications
ChatGPT 5.3, Plus	Implementing cron-based reminder processing in Next.js API routes	Enabled scheduled background execution for evaluating upcoming appointments and triggering reminders
ChatGPT 5.3, Plus	Designing multi-stage reminder system (5-day and 1-day reminders)	Created scalable reminder logic supporting multiple notification intervals
ChatGPT 5.3, Plus	Preventing duplicate reminder emails using database flags	Introduced reminder state tracking fields (reminder5dSent, reminder1dSent) to ensure idempotent notification behavior
ChatGPT 5.3, Plus	Debugging reminder logic and validating appointment time window calculations	Resolved scheduling edge cases and ensured accurate reminder triggering

ChatGPT 5.3, Plus	Generating large seed datasets for development and testing	Enabled efficient testing of scheduling, reminders, and role-based features using realistic test data
ChatGPT 5.3, Plus	Fixing Prisma seed script errors and type mismatches	Resolved TypeScript issues and ensured successful database seeding
ChatGPT 5.3, Plus	Debugging service worker and notification readiness issues	Verified correct service worker registration and ensured system readiness for notifications
ChatGPT 5.3, Plus	Validating reminder cron endpoint behavior during development	Confirmed correct execution of scheduled reminder jobs and email dispatch

6. Repo Check-in

Work on the repository was carried out on a regular basis, aligned with the project timeline and development milestones. Development activities included frontend UI components, backend API implementation, database schema evolution, PWA components and deployments.

Majority of the work for the eco-clean-web was committed through the branch **calendar and extend-notes**.

PR #10: https://github.com/vidarshan/W26_4495_S2_UpulA/pull/10

PR #11: https://github.com/vidarshan/W26_4495_S2_UpulA/pull/11

PR #12: https://github.com/vidarshan/W26_4495_S2_UpulA/pull/12

The work was checked into the repository almost every day, followed by a merge to main almost every 3 days to publish changes to the live preview.

